

ONLINE RETAIL SALES DATABASE DESIGN

Internship Project Report

Tools Used: PostgreSQL, lucid.chart

1. Project Objective

The objective of this project is to design and implement a fully normalized relational database schema for an electronic commerce platform.

The system supports:

- Customer management
- Product catalog management
- Category hierarchy (with subcategories)
- Order processing
- Payment tracking
- Sales reporting

The database is designed using Third Normal Form (3NF) to ensure:

- Elimination of redundancy
 - Enforcement of data integrity
 - Efficient querying
 - Scalability
-

2. System Overview

The system includes the following core entities:

- Customers
- Addresses
- Categories (with recursive hierarchy)
- Products
- Orders
- Order_Items (resolves many-to-many)
- Payment

3. Creating the Entities (Tables):

1. Customers

- first_name
- last_name
- email (unique)
- phone
- created_at

2. Addresses

- PK: address_id
- FK: customer_id
- street
- city
- state
- postal_code
- country

3. Categories

- PK: category_id
- category_name (unique)

4. Products

- PK: product_id
- FK: category_id
- product_name
- description
- price
- stock_quantity
- created_at

5. Orders

- PK: order_id
- FK: customer_id
- FK: shipping_address_id
- order_date
- status

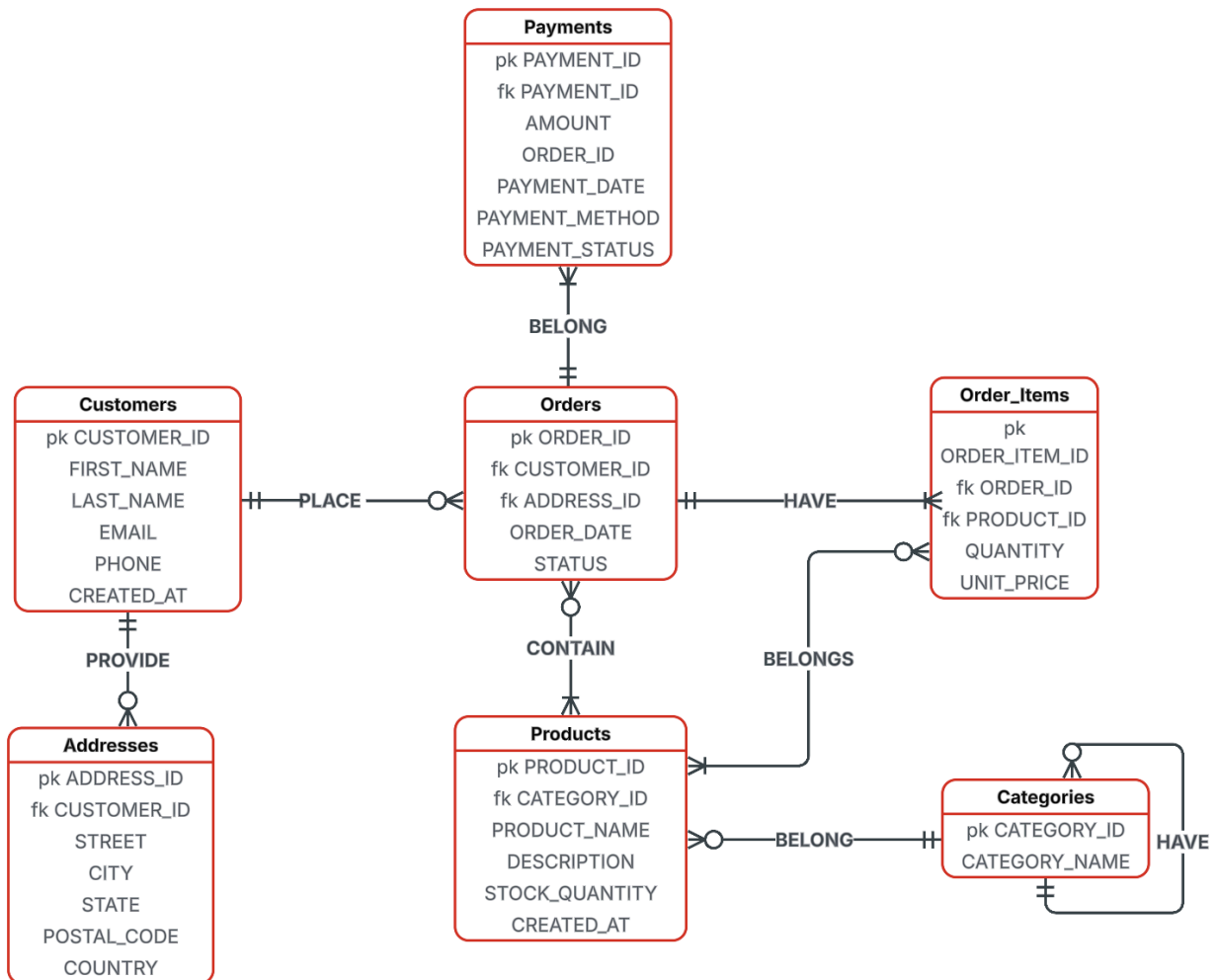
6. Order_Items

- PK: order_item_id
- FK: order_id
- FK: product_id
- quantity
- unit_price

7. Payments

- PK: payment_id
- FK: order_id (unique)
- payment_date
- payment_method
- Amount

4. Entity Relationship Design (ERD)



Main Relationships

1. Each category may have zero or many subcategories (0:N), and each subcategory must belong to exactly one parent category (1:1).
2. Each order must have at least one order_item (1:N) and each order_item must belong to exactly one order (1:1).
3. An order must contain at least one product (1:N), and a product may appear in zero or many order items (0:N).
4. Each category can contain zero or many products (0:N), while each product must belong to exactly one category (1:1).
5. Each order may be associated with at most one payment (0:1), and each payment must belong to exactly one order (1:1).
6. A customer may place zero or many orders (0:N), but each order must belong to exactly one customer (1:1).

7.A customer may provide zero or many addresses (0:N), and each address must belong to exactly one customer (1:1).

8.A product may appear in zero or many order items (0:N), while each order item must reference exactly one product (1:1).

5. Relational Models:

1. CUSTOMERS(CUSTOMER_ID, FIRST_NAME, LAST_NAME, EMAIL, PHONE, CREATED_AT)
 2. ADDRESSES(ADDRESS_ID, CUSTOMER_ID (fk), STREET, CITY, STATE, POSTAL_CODE, COUNTRY)
 3. ORDERS(ORDER_ID, CUSTOMER_ID (fk), PRODUCT_ID (fk), ORDER_DATE, STATUS)
 4. PAYMENTS(PAYMENT_ID, ORDER_ID (fk), AMOUNT, PAYMENT_DATE, PAYMENT_METHOD, PAYMENT_STATUS)
 5. CATEGORIES(CATEGORY_ID, CATEGORY_NAME, PARENT_CATEGORY_ID (fk))
 6. PRODUCTS(PRODUCT_ID, CATEGORY_ID (fk), ORDER_ID (fk), ORDER_ITEM_ID (fk), PRODUCT_NAME, DESCRIPTION, STOCK_QUANTITY, CREATED_AT)
 7. ORDER_ITEMS(ORDER_ITEM_ID, ORDER_ID (fk), PRODUCT_ID (fk), QUANTITY, UNIT_PRICE)
 8. PRODUCT_ORDER_ITEMS(PRODUCT_ID (fk), ORDER_ITEM_ID (fk))
 9. PRODUCT_ORDER(PRODUCT_ID (fk), ORDER_ID (fk))
-

5. Functional Dependency:

CUSTOMERS

CUSTOMERS(CUSTOMER_ID, FIRST_NAME, LAST_NAME, EMAIL, PHONE, CREATED_AT)

CUSTOMER_ID → FIRST_NAME, LAST_NAME, EMAIL, PHONE, CREATED_AT

EMAIL → CUSTOMER_ID, FIRST_NAME, LAST_NAME, PHONE, CREATED_AT

ADDRESSES

ADDRESSES(ADDRESS_ID, CUSTOMER_ID (fk), STREET, CITY, STATE, POSTAL_CODE, COUNTRY)

ADDRESS_ID → CUSTOMER_ID, STREET, CITY, STATE, POSTAL_CODE, COUNTRY

ORDERS

ORDERS(ORDER_ID, CUSTOMER_ID (fk), PRODUCT_ID (fk), ORDER_DATE, STATUS)

ORDER_ID → CUSTOMER_ID, PRODUCT_ID, ORDER_DATE, STATUS

PAYMENTS

PAYMENTS(PAYMENT_ID, ORDER_ID (fk), AMOUNT, PAYMENT_DATE, PAYMENT_METHOD, PAYMENT_STATUS)

PAYMENT_ID → ORDER_ID, AMOUNT, PAYMENT_DATE, PAYMENT_METHOD, PAYMENT_STATUS

ORDER_ID → PAYMENT_ID, AMOUNT, PAYMENT_DATE, PAYMENT_METHOD, PAYMENT_STATUS

CATEGORIES

CATEGORIES(CATEGORY_ID, CATEGORY_NAME, PARENT_CATEGORY_ID (fk))

CATEGORY_ID → CATEGORY_NAME, PARENT_CATEGORY_ID

PRODUCTS

PRODUCTS(PRODUCT_ID, CATEGORY_ID (fk), ORDER_ID (fk), ORDER_ITEM_ID (fk), PRODUCT_NAME, DESCRIPTION, STOCK_QUANTITY, CREATED_AT)

PRODUCT_ID → CATEGORY_ID, ORDER_ID, ORDER_ITEM_ID, PRODUCT_NAME, DESCRIPTION, STOCK_QUANTITY, CREATED_AT

ORDER_ITEMS

ORDER_ITEMS(ORDER_ITEM_ID, ORDER_ID (fk), PRODUCT_ID (fk), QUANTITY, UNIT_PRICE)

ORDER_ITEM_ID → ORDER_ID, PRODUCT_ID, QUANTITY, UNIT_PRICE

6. Normalization

1NF

- All attributes are atomic
- No repeating groups

2NF

- All tables use single-column primary keys
- No partial dependencies

3NF

- No transitive dependencies
- Non-key attributes depend only on primary key

Thus, the schema is fully normalized to **Third Normal Form (3NF)**.

1. CUSTOMERS

CUSTOMERS(CUSTOMER_ID, FIRST_NAME, LAST_NAME, EMAIL, PHONE, CREATED_AT)

EMAIL is duplicated

CUSTOMER_ID is primary key

1NF:

All attributes are atomic and No repeating groups. So it is in **1NF**.

FD1: CUSTOMER_ID → FIRST_NAME, LAST_NAME, EMAIL, PHONE, CREATED_AT

FD2: EMAIL → PHONE

2NF:

Since the primary key is a single attribute (CUSTOMER_ID), there is no partial dependency. It is already in **2NF**.

FD1: CUSTOMER_ID → FIRST_NAME, LAST_NAME, EMAIL, PHONE, CREATED_AT

FD2: EMAIL → PHONE,

3NF:

CUSTOMERS(CUSTOMER_ID, FIRST_NAME, LAST_NAME, EMAIL(fk), CREATED_AT)

FD1: CUSTOMER_ID → FIRST_NAME, LAST_NAME, EMAIL, CREATED_AT

CUSTOMER_DETAIL (EMAIL, PHONE)

FD1: EMAIL → PHONE

2. PAYMENTS

PAYMENTS(PAYMENT_ID, ORDER_ID (fk), AMOUNT, PAYMENT_DATE, PAYMENT_METHOD, PAYMENT_STATUS)

SINCE **ORDER_ID IS FOREIGN KEY**, then:

- Only PAYMENT_ID is the primary key

1NF:

All attributes are atomic and No repeating groups. So it is in **1NF**.

FD1: PAYMENT_ID → ORDER_ID, AMOUNT, PAYMENT_DATE, PAYMENT_METHOD,

PAYMENT_STATUS

FD2: PAYMENT_DATE → PAYMENT_METHOD, PAYMENT_STATUS

2NF:

Since the primary key is a single attribute (PAYMENT_ID), there is no partial dependency. It is already in **2NF**.

FD1: PAYMENT_ID → ORDER_ID, AMOUNT, PAYMENT_DATE, PAYMENT_METHOD,

PAYMENT_STATUS

FD2: PAYMENT_DATE → PAYMENT_METHOD, PAYMENT_STATUS

3NF:

PAYMENTS(PAYMENT_ID, ORDER_ID (fk), AMOUNT, PAYMENT_DATE(fk))

FD1: PAYMENT_ID → ORDER_ID, AMOUNT, PAYMENT_DATE

PAYMENT_TYPE(PAYMENT_DATE, PAYMENT_METHOD, PAYMENT_STATUS)

FD1: PAYMENT_DATE → PAYMENT_METHOD, PAYMENT_STATUS

4. ADDRESSES

ADDRESSES(ADDRESS_ID, CUSTOMER_ID (fk), STREET, CITY, STATE, POSTAL_CODE, COUNTRY)

1NF:

All attributes are atomic and No repeating groups. So it is in **1NF**.

FD1: ADDRESS_ID → CUSTOMER_ID, STREET, CITY, STATE, POSTAL_CODE, COUNTRY

2NF:

Since the primary key is a single attribute (ADDRESS_ID), there is no partial dependency. It is already in **2NF**.

FD1: ADDRESS_ID → CUSTOMER_ID, STREET, CITY, STATE, POSTAL_CODE, COUNTRY

3NF:

3NF requires: In 2NF AND No transitive dependency. So it is in **3NF**.

ADDRESSES(ADDRESS_ID, CUSTOMER_ID (fk), STREET, CITY, STATE, POSTAL_CODE, COUNTRY)

FD1: ADDRESS_ID → CUSTOMER_ID, STREET, CITY, STATE, POSTAL_CODE, COUNTRY

5. ORDERS

ORDERS(ORDER_ID, CUSTOMER_ID (fk), PRODUCT_ID (fk), ORDER_DATE, STATUS)

ORDER_ID → CUSTOMER_ID, PRODUCT_ID, ORDER_DATE, STATUS

1NF:

All attributes are atomic and No repeating groups. So it is in **1NF**.

FD1: ORDER_ID → CUSTOMER_ID, PRODUCT_ID, ORDER_DATE, STATUS

2NF:

Since the primary key is a single attribute (ORDER_ID), there is no partial dependency. It is already in **2NF**.

FD1: ORDER_ID → CUSTOMER_ID, PRODUCT_ID, ORDER_DATE, STATUS

3NF:

3NF requires: In 2NF AND No transitive dependency. So it is in **3NF**.

ORDERS(ORDER_ID, CUSTOMER_ID (fk), PRODUCT_ID (fk), ORDER_DATE, STATUS)

ORDER_ID → CUSTOMER_ID, PRODUCT_ID, ORDER_DATE, STATUS

6. CATEGORIES

CATEGORIES(CATEGORY_ID, CATEGORY_NAME, PARENT_CATEGORY_ID (fk))

FD1: CATEGORY_ID → CATEGORY_NAME, PARENT_CATEGORY_ID

1NF:

All attributes are atomic and No repeating groups. So it is in **1NF**.

FD1: CATEGORY_ID → CATEGORY_NAME, PARENT_CATEGORY_ID

2NF:

Since the primary key is a single attribute (CATEGORY_ID), there is no partial dependency. It is already in **2NF**.

FD1: CATEGORY_ID → CATEGORY_NAME, PARENT_CATEGORY_ID

3NF:

3NF requires: In 2NF AND No transitive dependency. So it is in **3NF**.

CATEGORIES(CATEGORY_ID, CATEGORY_NAME, PARENT_CATEGORY_ID (fk))

FD1: CATEGORY_ID → CATEGORY_NAME, PARENT_CATEGORY_ID

7. PRODUCTS

PRODUCTS(PRODUCT_ID, CATEGORY_ID (fk), ORDER_ID (fk), ORDER_ITEM_ID (fk),

PRODUCT_NAME, DESCRIPTION, STOCK_QUANTITY, CREATED_AT)

PRODUCT_ID → CATEGORY_ID, ORDER_ID, ORDER_ITEM_ID, PRODUCT_NAME, DESCRIPTION, STOCK_QUANTITY, CREATED_AT

1NF:

All attributes are atomic and No repeating groups. So it is in **1NF**.

FD1: PRODUCT_ID → CATEGORY_ID, ORDER_ID, ORDER_ITEM_ID, PRODUCT_NAME, DESCRIPTION, STOCK_QUANTITY, CREATED_AT

2NF:

Since the primary key is a single attribute (PRODUCT_ID), there is no partial dependency. It is already in **2NF**.

FD1: PRODUCT_ID → CATEGORY_ID, ORDER_ID, ORDER_ITEM_ID, PRODUCT_NAME, DESCRIPTION, STOCK_QUANTITY, CREATED_AT

3NF:

3NF requires: In 2NF AND No transitive dependency. So it is in **3NF**.

PRODUCTS(PRODUCT_ID, CATEGORY_ID (fk), ORDER_ID (fk), ORDER_ITEM_ID (fk),

PRODUCT_NAME, DESCRIPTION, STOCK_QUANTITY, CREATED_AT)

FD1: PRODUCT_ID → CATEGORY_ID, ORDER_ID, ORDER_ITEM_ID, PRODUCT_NAME, DESCRIPTION, STOCK_QUANTITY, CREATED_AT

8. ORDER_ITEMS

ORDER_ITEMS(ORDER_ITEM_ID, ORDER_ID (fk), PRODUCT_ID (fk), QUANTITY, UNIT_PRICE)

FD1: ORDER_ITEM_ID → ORDER_ID, PRODUCT_ID, QUANTITY, UNIT_PRICE

1NF:

All attributes are atomic and No repeating groups. So it is in **1NF**.

FD1: ORDER_ITEM_ID → ORDER_ID, PRODUCT_ID, QUANTITY, UNIT_PRICE

2NF:

Since the primary key is a single attribute (ORDER_ITEM_ID), there is no partial dependency. It is already in **2NF**.

FD1: ORDER_ITEM_ID → ORDER_ID, PRODUCT_ID, QUANTITY, UNIT_PRICE

3NF:

3NF requires: In 2NF AND No transitive dependency. So it is in **3NF**.

ORDER_ITEMS(ORDER_ITEM_ID, ORDER_ID (fk), PRODUCT_ID (fk), QUANTITY, UNIT_PRICE)

FD1: ORDER_ITEM_ID → ORDER_ID, PRODUCT_ID, QUANTITY, UNIT_PRICE

Based on the functional dependency analysis, all relations are in 1NF as attributes are atomic. Since all primary keys are single attributes, there are no partial dependencies; therefore, all tables are in 2NF. Additionally, there are no transitive dependencies between non-key attributes, which confirms that the schema is fully normalized to Third Normal Form (3NF).

ENUM TYPES

```
CREATE TYPE order_status_enum AS ENUM ('Pending', 'Shipped', 'Delivered', 'Cancelled');  
CREATE TYPE payment_status_enum AS ENUM ('Pending', 'Completed', 'Failed');
```

CUSTOMERS

```
CREATE TABLE customers (  
  customer_id SERIAL PRIMARY KEY,  
  first_name VARCHAR(50) NOT NULL,  
  last_name VARCHAR(50) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  phone VARCHAR(20),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

ADDRESSES

```
CREATE TABLE addresses (  
  address_id SERIAL PRIMARY KEY,  
  customer_id INT NOT NULL REFERENCES customers(customer_id) ON DELETE  
  CASCADE,  
  street VARCHAR(150) NOT NULL,  
  city VARCHAR(50) NOT NULL,  
  state VARCHAR(50),  
  postal_code VARCHAR(20),  
  country VARCHAR(50) NOT NULL  
);
```

CATEGORIES (With Recursive Hierarchy)

```
CREATE TABLE categories (  
  category_id SERIAL PRIMARY KEY,  
  category_name VARCHAR(100) UNIQUE NOT NULL,  
  parent_category_id INT REFERENCES categories(category_id)  
);
```

PRODUCTS

```
CREATE TABLE products (  
  product_id SERIAL PRIMARY KEY,  
  category_id INT NOT NULL REFERENCES categories(category_id) ON DELETE RESTRICT,  
  product_name VARCHAR(100) NOT NULL,  
  description TEXT,  
  price NUMERIC(10,2) NOT NULL CHECK (price > 0),  
  stock_quantity INT NOT NULL CHECK (stock_quantity >= 0),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

ORDERS

```
CREATE TABLE orders (  
  order_id SERIAL PRIMARY KEY,  
  customer_id INT NOT NULL REFERENCES customers(customer_id) ON DELETE  
CASCADE,  
  shipping_address_id INT NOT NULL REFERENCES addresses(address_id),  
  order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  status order_status_enum DEFAULT 'Pending'  
);
```

ORDER_ITEMS

```
CREATE TABLE order_items (  
  order_item_id SERIAL PRIMARY KEY,  
  order_id INT NOT NULL REFERENCES orders(order_id) ON DELETE CASCADE,  
  product_id INT NOT NULL REFERENCES products(product_id),  
  quantity INT NOT NULL CHECK (quantity > 0),  
  unit_price NUMERIC(10,2) NOT NULL CHECK (unit_price > 0),  
  UNIQUE(order_id, product_id)  
);
```

PAYMENTS

```
CREATE TABLE payments (  
  payment_id SERIAL PRIMARY KEY,  
  order_id INT UNIQUE NOT NULL REFERENCES orders(order_id) ON DELETE CASCADE,  
  payment_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  payment_method VARCHAR(50) NOT NULL,  
  amount NUMERIC(10,2) NOT NULL CHECK (amount > 0),  
  payment_status payment_status_enum DEFAULT 'Pending'  
);
```

7. Indexing Strategy

```
CREATE INDEX idx_orders_customer ON orders(customer_id);
CREATE INDEX idx_order_items_order ON order_items(order_id);
CREATE INDEX idx_order_items_product ON order_items(product_id);
CREATE INDEX idx_products_category ON products(category_id);
CREATE INDEX idx_payments_order ON payments(order_id);
```

Indexes improve:

- JOIN performance
 - Sales reporting
 - Dashboard query speed
-

8. Sample Data Population

-- Categories

```
INSERT INTO categories (category_name) VALUES
('Electronics'), ('Clothing');
```

-- Products

```
INSERT INTO products (category_id, product_name, description, price, stock_quantity)
VALUES
(1, 'Laptop', '14 inch business laptop', 800.00, 50),
(1, 'Headphones', 'Noise cancelling', 150.00, 100),
(2, 'T-Shirt', 'Cotton T-Shirt', 25.00, 200);
```

-- Customers

```
INSERT INTO customers (first_name, last_name, email, phone)
VALUES
('John', 'Doe', 'john@example.com', '1234567890'),
('Jane', 'Smith', 'jane@example.com', '0987654321');
```

-- Order Status

```
INSERT INTO order_status (status_name) VALUES
('Pending'), ('Shipped'), ('Delivered');
```

-- Payment Methods

```
INSERT INTO payment_methods (method_name) VALUES
('Credit Card'), ('PayPal');
```

-- Orders

```
INSERT INTO orders (customer_id, status_id, total_amount)
VALUES
(1, 3, 950.00),
(2, 1, 25.00);

-- Order Items
INSERT INTO order_items (order_id, product_id, quantity, unit_price)
VALUES
(1, 1, 1, 800.00),
(1, 2, 1, 150.00),
(2, 3, 1, 25.00);

-- Payments
INSERT INTO payments (order_id, payment_method_id, amount, payment_status)
VALUES
(1, 1, 950.00, 'Completed'),
(2, 2, 25.00, 'Pending');
```

9. Analytical Queries & Reports

Order Details Report

```
SELECT
o.order_id,
CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
o.order_date,
o.status,
SUM(oi.quantity * oi.unit_price) AS total_amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY o.order_id, c.first_name, c.last_name, o.order_date, o.status
ORDER BY o.order_date DESC;
```

Detailed Sales Report

```
SELECT
o.order_id,
p.product_name,
oi.quantity,
oi.unit_price,
(oi.quantity * oi.unit_price) AS line_total
```

```
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
JOIN products p ON oi.product_id = p.product_id;
```

Total Sales by Product

```
SELECT
  p.product_name,
  SUM(oi.quantity) AS total_units_sold,
  SUM(oi.quantity * oi.unit_price) AS total_revenue
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.product_name;
```

Sales Summary View

```
CREATE VIEW sales_summary AS
SELECT
  o.order_id,
  c.first_name,
  c.last_name,
  SUM(oi.quantity * oi.unit_price) AS total_order_value
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY o.order_id, c.first_name, c.last_name;
```

10. Conclusion

This project successfully implements:

- A fully normalized 3NF relational schema
- Recursive category hierarchy
- Proper resolution of many-to-many relationships
- Data integrity via constraints
- Performance optimization using indexes
- Analytical reporting using JOIN queries and views

The system is scalable, consistent, and optimized for real world e-commerce operations
